```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * File Name          : freertos.c
  * Description        : Code for freertos applications
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Includes ------------------------------------------------------------------*/
#include "FreeRTOS.h"
#include "task.h"
#include "main.h"
#include "cmsis_os.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h> // For strlen() and standard string
```

```
handling
#include <stdio.h>  // For sprintf() formatting
tracking utilities

// Hardware Handle Linkage from main.c
extern TIM_HandleTypeDef htim1;
extern TIM_HandleTypeDef htim3;
extern TIM_HandleTypeDef htim5;
extern UART_HandleTypeDef huart1;
extern SPI_HandleTypeDef hspi2;
extern RTC_HandleTypeDef hrtc;
extern void SystemClock_Config();


// Shared Hardware State Variables from main.c
extern uint32_t lastBlinkTime;
extern uint8_t  ledIsOn;
extern uint32_t duty_cycle;
extern uint8_t  k0_last_state;
extern uint8_t  k1_last_state;

// Single Source-of-Truth LoRaWAN Frame Counter Variable
extern uint16_t frame_counter;
/* USER CODE END Includes */

/* Private typedef
-----------------------------------------------------------
---*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define
-------------------------------------------------------------
----*/
/* USER CODE BEGIN PD */
// LoRa RF95 Transceiver Register Map Definitions
```

```
#define REG_FIFO                    0x00
#define REG_OP_MODE                 0x01
#define REG_FRF_MSB                 0x06
#define REG_FRF_MID                 0x07
#define REG_FRF_LSB                 0x08
#define REG_PA_CONFIG               0x09
#define REG_FIFO_ADDR_PTR           0x0D
#define REG_FIFO_TX_BASE_ADDR       0x0E
#define REG_MODEM_CONFIG_1          0x1D
#define REG_MODEM_CONFIG_2          0x1E
#define REG_PAYLOAD_LENGTH          0x22
#define REG_DIO_MAPPING_1           0x40


// Radio Functional Operational Modes
#define MODE_LONG_RANGE_MODE        0x80
#define MODE_SLEEP                  0x00
#define MODE_STDBY                  0x01
#define MODE_TX                     0x03
#define MODE_RX_SINGLE              0x06

// Legacy Alias Support
#define RF95_REG_FIFO               REG_FIFO
#define RF95_REG_OP_MODE            REG_OP_MODE
#define RF95_MODE_TX                MODE_TX
/* USER CODE END PD */

/* Private macro
-------------------------------------------------------
-----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables
-------------------------------------------------------
-*/
```

```
/* USER CODE BEGIN Variables */
typedef enum {
    STATE_INIT,
    STATE_TX,
    STATE_RX1,
    STATE_RX2,
    STATE_SLEEP
} LoraState_t;

static LoraState_t current_state = STATE_INIT;

typedef enum {
    STATE_IDLE,
    STATE_WAITING_FOR_TX,
    STATE_WAITING_FOR_RX
} DIO0_Line_State_t;

volatile DIO0_Line_State_t DIO0_Line_current_state =
STATE_IDLE;

const uint32_t eu868_channels[3] = {868100000, 868300000
, 868500000};
static uint8_t channel_index = 0;
uint8_t lora_rx_packet_buffer[256];

/* USER CODE END Variables */
/* Definitions for MainLogicTask */
osThreadId_t MainLogicTaskHandle;
const osThreadAttr_t MainLogicTask_attributes = {
  .name = "MainLogicTask",
  .stack_size = 1536 * 4,
  .priority = (osPriority_t) osPriorityNormal,
};
/* Definitions for SerialTask */
osThreadId_t SerialTaskHandle;
const osThreadAttr_t SerialTask_attributes = {
  .name = "SerialTask",
```

```
  .stack_size = 1536 * 4,
  .priority = (osPriority_t) osPriorityBelowNormal,
};
/* Definitions for LoRaTask */
osThreadId_t LoRaTaskHandle;
const osThreadAttr_t LoRaTask_attributes = {
  .name = "LoRaTask",
  .stack_size = 1536 * 4,
  .priority = (osPriority_t) osPriorityHigh,
};

/* Private function prototypes
-----------------------------------------------*/
/* USER CODE BEGIN FunctionPrototypes */
// Core Radio Driver Commands
void Lora_WriteReg(uint8_t addr, uint8_t val);
uint8_t Lora_ReadReg(uint8_t addr);
void Lora_ReadFIFOVerify(uint8_t *buffer, uint8_t length)
;
void Lora_SetFrequency(uint32_t frequency_hz);
void LoRa_Extract_FIFO_Payload(uint8_t *rx_buffer,
uint8_t *payload_length);

// Wear-Leveled Flash Counter Module Drivers
uint16_t Load_Frame_Counter_Wear_Leveled(void);
void Save_Frame_Counter_Wear_Leveled(uint16_t counter);

// LoRaWAN Cryptographic Libraries Linkage
extern void aes_encrypt_block(const uint8_t *key, const
 uint8_t *input, uint8_t *output);
extern void calculate_lorawan_mic(uint8_t *packet,
uint8_t payload_len, const uint8_t *nwkSKey, uint8_t *
mic_out);
extern void encrypt_lorawan_payload(uint8_t *payload,
uint8_t payload_len, const uint8_t *appSKey, uint32_t
devAddr, uint16_t fCnt);
/* USER CODE END FunctionPrototypes */
```

```c
void StartMainLogicTask(void *argument);
void StartSerialTask(void *argument);
void StartLoRaTask(void *argument);

void MX_FREERTOS_Init(void); /* (MISRA C 2004 rule 8.1) */

/* Hook prototypes */
void vApplicationStackOverflowHook(xTaskHandle xTask, signed char *pcTaskName);

/* USER CODE BEGIN 4 */
void vApplicationStackOverflowHook(xTaskHandle xTask, signed char *pcTaskName)
{
   /* Run time stack overflow checking is performed if
   configCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2. This hook function is
   called if a stack overflow is detected. */
   for(;;); // Trap context execution here for debugging stack breaks
}
/* USER CODE END 4 */

/**
  * @brief  FreeRTOS initialization
  * @param  None
  * @retval None
  */
void MX_FREERTOS_Init(void) {
  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* USER CODE BEGIN RTOS_MUTEX */
  /* add mutexes, ... */
```

```
  /* USER CODE END RTOS_MUTEX */

  /* USER CODE BEGIN RTOS_SEMAPHORES */
  /* add semaphores, ... */
  /* USER CODE END RTOS_SEMAPHORES */

  /* USER CODE BEGIN RTOS_TIMERS */
  /* start timers, add new ones, ... */
  /* USER CODE END RTOS_TIMERS */

  /* USER CODE BEGIN RTOS_QUEUES */
  /* add queues, ... */
  /* USER CODE END RTOS_QUEUES */

  /* Create the thread(s) */
  /* creation of MainLogicTask */
  MainLogicTaskHandle = osThreadNew(StartMainLogicTask,
  NULL, &MainLogicTask_attributes);

  /* creation of SerialTask */
  SerialTaskHandle = osThreadNew(StartSerialTask, NULL,
  &SerialTask_attributes);

  /* creation of LoRaTask */
  LoRaTaskHandle = osThreadNew(StartLoRaTask, NULL, &
  LoRaTask_attributes);

  /* USER CODE BEGIN RTOS_THREADS */
  /* add threads, ... */
  /* USER CODE END RTOS_THREADS */

  /* USER CODE BEGIN RTOS_EVENTS */
  /* add events, ... */
  /* USER CODE END RTOS_EVENTS */

}
```

```
/* USER CODE BEGIN Header_StartMainLogicTask */
/**
  * @brief  Function implementing the MainLogicTask thread.
  * @param  argument: Not used
  * @retval None
  */
/* USER CODE END Header_StartMainLogicTask */
void StartMainLogicTask(void *argument)
{
  /* USER CODE BEGIN StartMainLogicTask */
  /* Infinite loop */
  for(;;)
  {
    // 1. BLINK LOGIC (Non-blocking)
    if (HAL_GetTick() - lastBlinkTime >= 500)
    {
        lastBlinkTime = HAL_GetTick();
        ledIsOn = !ledIsOn; // Flip the state

        if (ledIsOn) {
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
             duty_cycle);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2,
            0);
        } else {
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
            0);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2,
             duty_cycle);
        }
    }

    // Check K0: Increase Duty
    uint8_t k0_current = HAL_GPIO_ReadPin(GPIOE,
    GPIO_PIN_4);
    if (k0_current == 0 && k0_last_state == 1) {
```

```
        if (duty_cycle <= 950) duty_cycle += 50;
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1,
        duty_cycle); // updates PE9
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
        duty_cycle); // Updates Onboard LED
        printf("Duty Increased: %lu\r\n", duty_cycle);
        osDelay(50); // Debounce
    }
    k0_last_state = k0_current;

    // Check K1: Decrease Duty
    uint8_t k1_current = HAL_GPIO_ReadPin(GPIOE,
    GPIO_PIN_3);
    if (k1_current == 0 && k1_last_state == 1) {
        if (duty_cycle >= 50) duty_cycle -= 50;
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1,
        duty_cycle); // updates PE9
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
        duty_cycle); // Updates Onboard LED
        printf("Duty Decreased: %lu\r\n", duty_cycle);
        osDelay(50); // Debounce
    }
    k1_last_state = k1_current;


    osDelay(10);
  }
  /* USER CODE END StartMainLogicTask */
}

/* USER CODE BEGIN Header_StartSerialTask */
/**
* @brief Function implementing the SerialTask thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_StartSerialTask */
```

```c
void StartSerialTask(void *argument)
{
  /* USER CODE BEGIN StartSerialTask */
  /* Infinite loop */
  for(;;)
  {
    char msg[] = "RTOS Heartbeat: System Stable\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg
    ), 100);
    osDelay(2000); // Send a message every 2 seconds
  }
  /* USER CODE END StartSerialTask */
}

/* USER CODE BEGIN Header_StartLoRaTask */
/**
* @brief Function implementing the LoRaTask thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_StartLoRaTask */
void StartLoRaTask(void *argument)
{
  /* USER CODE BEGIN StartLoRaTask */

  // secret keys recovered from your dashboard session
  const uint8_t nwkSKey[16] = { 0x15, 0x95, 0x75, 0x32,
  0x81, 0xC7, 0x92, 0x28, 0x75, 0x4D, 0x76, 0xA0, 0x1D,
  0xA2, 0xFD, 0x7C };
  const uint8_t appSKey[16] = { 0x29, 0x93, 0x62, 0x1E,
  0x0C, 0x03, 0xE1, 0xCA, 0x6B, 0x28, 0x45, 0x81, 0x40,
  0x45, 0xFB, 0x46 };

  uint8_t packet[64];
  uint8_t idx = 0;
  uint8_t irqFlags = 0;
  uint8_t rx_base;
```

```
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
  /*
```

```
  /*
  ========================================================
  ===
   * 🏭 STAGE A: ONE-TIME BOOT UP HARDWARE VALIDATION
   *
   ========================================================
   ==== */
  HAL_GPIO_WritePin(LORA_RESET_GPIO_Port, LORA_RESET_Pin,
  GPIO_PIN_RESET);
  osDelay(10); // 10 ms delay
  HAL_GPIO_WritePin(LORA_RESET_GPIO_Port, LORA_RESET_Pin,
  GPIO_PIN_SET);
  osDelay(10); // 10 ms delay (requirement is at least
  5ms before accessing chip

  // Confirm SPI Interface is working
  uint8_t version = Lora_ReadReg(0x42);
  if (version != 0x12) {
      HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa]
      ERROR: Radio core not found!\r\n", 37, 100);
      for(;;) { osDelay(1000); }
  }

  // Baseline modem register alignments
  Lora_WriteReg(REG_OP_MODE, MODE_LONG_RANGE_MODE |
  MODE_SLEEP);  // put into sleep mode
  osDelay(10);

  Lora_WriteReg(REG_FIFO_TX_BASE_ADDR, 0x00); // Reset
  FIFOs
  Lora_WriteReg(REG_FIFO_ADDR_PTR, 0x00);     // Reset
  FIFOs
  Lora_WriteReg(REG_MODEM_CONFIG_1, 0x70);    //
  125kHz, CR 4/5
  Lora_WriteReg(REG_MODEM_CONFIG_2, 0x74);    // SF7,
  CRC On
  Lora_WriteReg(REG_PA_CONFIG, 0x80);         // low
```

```
  power for ttg-68 in same room - was 0x8F (PA_BOOST
  Enabled, Max Power)

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*

  /*
```

```
  /*
  ================================================================
  ===
   * 🔄 STAGE B: THE CORE OS STATE ENGINE LOOP
   *
   ================================================================
   ==== */
  for(;;)
  {
      switch(current_state) {

          case STATE_INIT:
              HAL_UART_Transmit(&huart1, (uint8_t*)
              "\r\n[LoRa] -> State: INIT. Staging
              payload...\r\n", 46, 100);

              // Dynamic cleanup resets to avoid
              registration lockups
              Lora_WriteReg(REG_OP_MODE,
              MODE_LONG_RANGE_MODE | MODE_STDBY);
              osDelay(5);
              Lora_WriteReg(0x12, 0xFF); // Clear all
              stale flags

              // Pull frequency from your dynamic
              channel array
              uint32_t active_tx_frequency =
              eu868_channels[channel_index];
              Lora_SetFrequency(active_tx_frequency);

              // Align operational modes for Transmission
              Lora_WriteReg(REG_DIO_MAPPING_1, 0x40);
              / Map DIO0 to TxDone
              Lora_WriteReg(0x33, 0x27);
              / Normal IQ for TX
              Lora_WriteReg(0x3B, 0x1D);
              / RegInvertIq2 standard default <-- ADDED
```

```
                FOR SX1276 RESET

                // Assemble the exact framework
                idx = 0;
                uint8_t dev_addr[4] = { 0x2D, 0x61, 0x00,
                0x27 }; // 2700612D LSB Array format

                packet[idx++] = 0x40;         // MHDR:
                Confirmed Uplink Type (Forced Gateway ACK
                Request)
                packet[idx++] = dev_addr[0];
                packet[idx++] = dev_addr[1];
                packet[idx++] = dev_addr[2];
                packet[idx++] = dev_addr[3];
                packet[idx++] = 0x00;         // FCtrl
                packet[idx++] = (frame_counter & 0xFF);

                packet[idx++] = ((frame_counter >> 8) &
                0xFF);
                packet[idx++] = 0x01;         // FPort

                // Staging application tracking structures
                uint8_t app_payload[16];
                strcpy((char*)app_payload, "HELLO1");
                uint8_t app_payload_len = strlen((char*)
                app_payload);

                // Encrypt data inline
                encrypt_lorawan_payload(app_payload,
                app_payload_len, appSKey, *(uint32_t*)
                dev_addr, frame_counter);

                // Fix: Append matching exact variable
                lengths dynamically
                for(uint8_t i = 0; i < app_payload_len; i+
                +) {
                    packet[idx++] = app_payload[i];
```

```
            }

            // Evaluate MIC Signature
            uint8_t payload_len_before_mic = idx;
            uint8_t real_mic[4];
            calculate_lorawan_mic(packet,
            payload_len_before_mic, nwkSKey, real_mic);

            packet[idx++] = real_mic[0];
            packet[idx++] = real_mic[1];
            packet[idx++] = real_mic[2];
            packet[idx++] = real_mic[3];

            // Push array payload to hardware FIFO
            Lora_WriteReg(REG_FIFO_ADDR_PTR, 0x00);
            Lora_WriteReg(REG_PAYLOAD_LENGTH, idx);
            for(uint8_t i = 0; i < idx; i++) {
                Lora_WriteReg(REG_FIFO, packet[i]);
            }

            // --- FIFO Verification Diagnostic Block
            ---
            Lora_WriteReg(REG_FIFO_ADDR_PTR, 0x00);
            uint8_t readback_verify[64] = {0};
            Lora_ReadFIFOVerify(readback_verify, idx);

            char uart_buf[100];
            HAL_UART_Transmit(&huart1, (uint8_t*)"---
             [LoRa FIFO Check] ---\r\n", 27, 100);
            for(uint8_t i = 0; i < idx; i++) {
                sprintf(uart_buf, "Byte [%02d]:
                Written=0x%02X | RadioHas=0x%02X\r\n",
                 i, packet[i], readback_verify[i]);
                HAL_UART_Transmit(&huart1, (uint8_t*)
                uart_buf, strlen(uart_buf), 100);
            }
            HAL_UART_Transmit(&huart1, (uint8_t*)"---
```

```
            ----------------------\r\n", 29, 100);

            Lora_WriteReg(REG_FIFO_ADDR_PTR, 0x00);
            / Re-zero tracking index
            //
            ----------------------------------------

            HAL_UART_Transmit(&huart1, (uint8_t*)
            "[LoRa] -> Broadcasting Styled
            Frame...\r\n", 39, 100);

            // Shift state and fire
            current_state = STATE_TX;

            DIO0_Line_current_state =
            STATE_WAITING_FOR_TX;

            Lora_WriteReg(REG_OP_MODE,
            MODE_LONG_RANGE_MODE | MODE_TX); //
            actually transmit packet


            break;


        case STATE_TX:
            // 1. Synchronously wait for the physical
            transmission to clear the airwaves (PE0 /
            EXTI0)
            ulTaskNotifyTake(pdTRUE, portMAX_DELAY);


            // 3. Acknowledge and clear the hardware
            radio TX flag
            Lora_WriteReg(0x12, 0x08);

            //HAL_UART_Transmit(&huart1,
```

```
                (uint8_t*)"[LoRa] -> State: TX Done. Scheduling
                low-power 980ms wait...\r\n", 61, 100);

                // 4. Shift modem parameters over to
                Receive mappings while in Standby
                (Standby mode is entered in automatically
                by the SX1276 when the irq TXDone is
                received
                Lora_WriteReg(REG_DIO_MAPPING_1, 0x00);
                / Re-map DIO0 to watch for RxDone
                Lora_WriteReg(0x33, 0x66);
                / Invert IQ for gateway synchronization
                Lora_WriteReg(0x3B, 0x19);
                / Invert IQ 2 (SX1276 critical patch)

                // 5. FIX: Hard-write 0x1E instead of
                read-modifying to prevent Spreading
                Factor corruption.
                // 0x73 = Spreading Factor 7 (0x70) | No
                continuous mode (0x00) | No Rx CRC (0x00)
                | SymbTimeout MSB (0x00)
                Lora_WriteReg(0x1E, 0x70);
                Lora_WriteReg(0x1F, 0x10);

                // 6. FIX: Reset the internal FIFO
                pointer to the RX base address before
                listening
                rx_base = Lora_ReadReg(0x0F);
                / Read RegFifoRxBaseAddr
                Lora_WriteReg(0x0D, rx_base);
                / Reset RegFifoAddrPtr to base point

                // 7. THE 980ms FIRST SLEEP PHASE: Wait
                for the RTC alarm clock to ring

                // Snapshot the total elapsed time
                (includes context switch + all SPI
```

```
            operations above)
            uint32_t elapsed_processing_time_us =
            __HAL_TIM_GET_COUNTER(&htim5);
            HAL_TIM_Base_Stop(&htim5); // Stop
            stopwatch

            const uint32_t TARGET_WINDOW_US = 1000000;
                // 1.000000 second target
            const uint32_t STM32F4_WAKEUP_LATENCY_US =
            30;  // Datasheet constant

            // Calculate exact remaining deep-sleep
            duration needed
            uint32_t actual_sleep_needed_us =
            TARGET_WINDOW_US -
            elapsed_processing_time_us -
            STM32F4_WAKEUP_LATENCY_US;

            // Convert microseconds to precise LSE
            2048Hz ticks
            uint32_t dynamic_rtc_ticks = (
            actual_sleep_needed_us * 2048) / 1000000;

        //        if (HAL_RTCEx_SetWakeUpTimer_IT(&hrtc,
        dynamic_rtc_ticks, RTC_WAKEUPCLOCK_RTCCLK_DIV16)
        != HAL_OK)
        //        {
        //            Error_Handler();
        //        }

            if (HAL_RTCEx_SetWakeUpTimer_IT(&hrtc,
            2047, RTC_WAKEUPCLOCK_RTCCLK_DIV16) !=
            HAL_OK)
            {
                Error_Handler();
            }
```

```
        // Clear all pending flags before
        sleeping
    __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(&hrtc,
    RTC_FLAG_WUTF);
    __HAL_RTC_WAKEUPTIMER_EXTI_CLEAR_FLAG();
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);

    vTaskSuspendAll();
    HAL_SuspendTick(); // <-- ADD THIS to
    stop the 1ms system timer from waking the
    CPU

    uint32_t systick_ctrl_backup = SysTick->
    CTRL;
    SysTick->CTRL &= ~(SysTick_CTRL_ENABLE_Msk
    | SysTick_CTRL_TICKINT_Msk);

    __disable_irq();

    HAL_PWR_EnterSTOPMode(
    PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI
    );

    //

    //

    //

    //

    //

    //

    //
```

```
                //
                ========================================
                ========================
                // WAKEUP EVENT 1: The 980ms RTC Alarm
                expires! The MCU resumes here.
                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //
```

```
                //
                ==========================================
                ========================
                SystemClock_Config();

                // --- RESTORE SYSTICK PERIPHERAL ---
                SysTick->CTRL = systick_ctrl_backup;

                __enable_irq();
                HAL_ResumeTick();
                xTaskResumeAll();

                HAL_RTCEx_DeactivateWakeUpTimer(&hrtc);

                 // 7. Strobe the radio into active
                 single-receive mode
                Lora_WriteReg(REG_OP_MODE,
                MODE_LONG_RANGE_MODE | MODE_RX_SINGLE);

                // drive debug pin low
                HAL_GPIO_WritePin(Debug_IO_GPIO_Port,
                Debug_IO_Pin, GPIO_PIN_RESET);
                // drive timing pin low
                HAL_GPIO_WritePin(Timing_IO_GPIO_Port,
                Timing_IO_Pin, GPIO_PIN_SET);

                // Update state so the next DIO0
                interrupt is processed as an RX packet
                DIO0_Line_current_state =
                STATE_WAITING_FOR_RX;


                HAL_UART_Transmit(&huart1, (uint8_t*)
                "[LoRa] -> RTC Alert! Opening RX1 window
                precisely on time...\r\n", 61, 100);
```

```
            // --- NEW DIAGNOSTIC READBACK BLOCK ---
            uint8_t verified_mode = Lora_ReadReg(0x01);
            // Read REG_OP_MODE
            uint8_t verified_flags = Lora_ReadReg(0x12)
            ; // Read REG_IRQ_FLAGS

            char debug_msg[128];
            int debug_len = snprintf(debug_msg, sizeof
            (debug_msg),
                                     "[LoRa
                                     Diagnostic] Mode
                                     Reg: 0x%02X |
                                     IRQ Flags:
                                     0x%02X\r\n",
                                     verified_mode,
                                     verified_flags);

            HAL_UART_Transmit(&huart1, (uint8_t*)
            debug_msg, debug_len, 100);
            // ------------------------------------

            // Move state pointer forward to the
            receive state
            current_state = STATE_RX1;
            break;

        case STATE_RX1:
           // 1. THE SECOND SLEEP PHASE: Safely lock
           the MCU core down *inside* the state itself
           vTaskSuspendAll();
           HAL_PWR_EnterSTOPMode(
           PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

           //

           //
```

```
            //
            ==========================================
            ======================
            // WAKEUP EVENT 2: Radio physically asserts
            PE0 (RxDone) or PE1 (RxTimeout)!
            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //
```

```
            //
            ==========================================
            ======================
            SystemClock_Config(); // Restore full CPU
            operational speeds
            HAL_ResumeTick();


    // FIX 1: Allow the analog PLL crystal circuits a
    clean moment to lock
    HAL_Delay(5);

    // FIX 2: Force a refresh of the peripheral clocks
    (especially SPI and GPIO)
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_SPI2_CLK_ENABLE();

    xTaskResumeAll();
    HAL_RTCEx_DeactivateWakeUpTimer(&hrtc);



            // 2. Consume the pending FreeRTOS task
            notification variables cleanly
            ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

            // 3. Capture and reset radio flags
            immediately to release the physical
            hardware lines
            irqFlags = Lora_ReadReg(0x12);
            Lora_WriteReg(0x12, 0xFF);

            // 4. Evaluate matching metrics
```

```
            (PayloadReady is High, PayloadCrcError is Low)
            if ((irqFlags & 0x40) && !(irqFlags & 0x20))
            {
                HAL_UART_Transmit(&huart1, (uint8_t*)
                "[LoRa] -> State: RX1 SUCCESS! Valid
                ACK packet caught.\r\n", 56, 100);

                uint8_t payload_length = Lora_ReadReg(
                0x13); // RegRxNbBytes

                if (payload_length > 0 && payload_length
                <= 256)
                {
                    uint8_t current_fifo_addr =
                    Lora_ReadReg(0x10); //
                    RegFifoRxCurrentAddr
                    Lora_WriteReg(0x0D, current_fifo_addr
                    );          // RegFifoAddrPtr

                    for (uint8_t i = 0; i <
                    payload_length; i++)
                    {
                        lora_rx_packet_buffer[i] =
                        Lora_ReadReg(0x00); // RegFifo
                    }

                    char hex_print_buf[128];
                    int len = snprintf(hex_print_buf,
                    sizeof(hex_print_buf), "[LoRa]
                    Extracted %d bytes: ", payload_length
                    );
                    HAL_UART_Transmit(&huart1, (uint8_t*)
                    hex_print_buf, len, 100);

                    for (uint8_t i = 0; i <
                    payload_length; i++)
                    {
```

```
                    len = snprintf(hex_print_buf,
                    sizeof(hex_print_buf), "%02X ",
                    lora_rx_packet_buffer[i]);
                    HAL_UART_Transmit(&huart1, (
                    uint8_t*)hex_print_buf, len, 100)
                    ;
                }
                HAL_UART_Transmit(&huart1, (uint8_t*)
                "\r\n", 2, 100);
            }
            // Return to Sleep to wait for next
            transmit
            Lora_WriteReg(0x01, 0x80 | 0x01);
            / Force Standby mode
            rx_base = Lora_ReadReg(0x0F);
            / Read RegFifoRxBaseAddr
            Lora_WriteReg(0x0D, rx_base);
            / Reset RegFifoAddrPtr to base point
            current_state = STATE_SLEEP;
        }
        else if (irqFlags & 0x20)
        {
            HAL_UART_Transmit(&huart1, (uint8_t*)
            "[LoRa] -> State: RX1 Packet caught but
            failed CRC.\r\n", 52, 100);

            // Handle CRC failure by routing to
            sleep normally
            Lora_WriteReg(0x01, 0x80 | 0x01);
            / Force Standby mode
            rx_base = Lora_ReadReg(0x0F);
            / Read RegFifoRxBaseAddr
            Lora_WriteReg(0x0D, rx_base);
            / Reset RegFifoAddrPtr to base point
            current_state = STATE_RX2;
        }
        else
```

```
            {
                // --- ROUTE TIMEOUT TO RX2 INSTEAD OF
                SLEEP ---
                HAL_UART_Transmit(&huart1, (uint8_t*)
                "[LoRa] -> RX1 Timeout. Pivoting to RX2
                (869.525MHz, SF9)...\r\n", 59, 100);

                // Move to Standby briefly to change
                config registers safely
                Lora_WriteReg(0x01, 0x80 | 0x01);

                // Reprogram to mandatory EU868 RX2
                fixed frequency
                Lora_SetFrequency(869525000);

                // Set Spreading Factor 9 (SF9) for RX2
                window
                Lora_WriteReg(0x1E, 0x93);
                Lora_WriteReg(0x1F, 0xFF);

                // Reset FIFO memory pointers to base
                RX block
                uint8_t rx_base_addr = Lora_ReadReg(0x0F)
                ;
                Lora_WriteReg(0x0D, rx_base_addr);

                // Strobe the radio receiver into
                active single-receive mode
                Lora_WriteReg(REG_OP_MODE,
                MODE_LONG_RANGE_MODE | MODE_RX_SINGLE);
                HAL_Delay(2); // Let fractional-N
                synthesizer lock onto 869.525MHz

                // Update state machine pointer to look
                at the new block next loop
                current_state = STATE_RX2;
            }
```

```
            DIO0_Line_current_state = STATE_IDLE;

            break; // Master break cleanly exits
            STATE_RX1

        case STATE_RX2:
            // 1. THE LOW-POWER WAITING PHASE:
            Suspend task ticks and sleep the STM32F4
            core
            vTaskSuspendAll();
            HAL_PWR_EnterSTOPMode(
            PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI
            );

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //

            //
```

```
                //
                ========================================
                ========================
                // PHYSICAL WAKEUP: PE0 (RxDone) or PE1
                (RxTimeout) asserts for RX2!
                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //

                //
```

```
                //
                ==========================================
                ========================
                SystemClock_Config(); // Instantly
                restore 168MHz HSE + PLL clock trees
                HAL_ResumeTick();
                xTaskResumeAll();

                // 2. Safely consume the pending FreeRTOS
                task notification flag
                ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

                // 3. Read and clear active radio flags
                to drop physical PE0/PE1 lines
                irqFlags = Lora_ReadReg(0x12);
                Lora_WriteReg(0x12, 0xFF);

                // 4. Evaluate RX2 packet metrics
                (PayloadReady is High, PayloadCrcError is
                Low)
                if ((irqFlags & 0x40) && !(irqFlags & 0x20
                ))
                {
                    HAL_UART_Transmit(&huart1, (uint8_t*)
                    "[LoRa] -> State: RX2 SUCCESS! Valid
                    Downlink Caught.\r\n", 55, 100);

                    uint8_t payload_length = Lora_ReadReg(
                    0x13); // RegRxNbBytes

                    if (payload_length > 0 &&
                    payload_length <= 256)
                    {
                        uint8_t current_fifo_addr =
                        Lora_ReadReg(0x10);
                        Lora_WriteReg(0x0D,
                        current_fifo_addr);
```

```
                for (uint8_t i = 0; i <
                payload_length; i++) {
                    lora_rx_packet_buffer[i] =
                    Lora_ReadReg(0x00);
                }

                char hex_print_buf[128];
                int len = snprintf(hex_print_buf,
                sizeof(hex_print_buf), "[LoRa
                RX2] Extracted %d bytes: ",
                payload_length);
                HAL_UART_Transmit(&huart1, (uint8_t
                *)hex_print_buf, len, 100);

                for (uint8_t i = 0; i <
                payload_length; i++) {
                    len = snprintf(hex_print_buf,
                    sizeof(hex_print_buf), "%02X ",
                    lora_rx_packet_buffer[i]);
                    HAL_UART_Transmit(&huart1, (
                    uint8_t*)hex_print_buf, len,
                    100);
                }
                HAL_UART_Transmit(&huart1, (uint8_t
                *)"\r\n", 2, 100);
            }
        }
        // 5. Handle standard RX2 window timeout
        or frame corruption closures
        else
        {
            HAL_UART_Transmit(&huart1, (uint8_t*)
            "[LoRa] -> State: RX2 Closed
            (Timeout/No Response).\r\n", 52, 100);
        }
```

```
            // 6. HARDWARE CLEANUP: Put the radio
            back to standby mode
            Lora_WriteReg(0x01, 0x80 | 0x01);

            // 7. MANDATORY CRITICAL PROTOCOL FIX:
            Re-read the active dynamic channel
            frequency
            // and update the radio register right
            now. If you omit this, your next
            transmission
            // will accidentally broadcast on
            869.525MHz, resulting in network drops.
            active_tx_frequency = eu868_channels[
            channel_index];
            Lora_SetFrequency(active_tx_frequency);

            // Restore Spreading Factor 7 default
            settings for your upcoming transmission
            sequences
            Lora_WriteReg(0x1E, 0x73);

            // 8. Safely advance to your 30-second
            low-power cooldown state
            current_state = STATE_SLEEP;
            break;



        case STATE_SLEEP:
            HAL_UART_Transmit(&huart1, (uint8_t*)
            "[LoRa] -> State: SLEEP. Incrementing
            frame counter an powering down radio for
            30s...\r\n", 56, 100);


            // Save current counter to maintain
```

```
                timeline accuracy
                Save_Frame_Counter_Wear_Leveled(
                frame_counter);
                frame_counter++;

                // Drop radio core down to micro-amps
                Lora_WriteReg(REG_OP_MODE,
                MODE_LONG_RANGE_MODE | MODE_SLEEP);

                // Cycle channel pointer for the upcoming
                iteration loop
                channel_index++;
                if (channel_index >= 3) {
                    channel_index = 0;
                }

                // Sleep the FreeRTOS task completely
                (set to 30s loop interval for verification)
                osDelay(pdMS_TO_TICKS(120000UL));

                // Return to init phase to restart
                hands-free
                current_state = STATE_INIT;
                break;
            }
        }
        /* USER CODE END StartLoRaTask */
}

/* Private application code
---------------------------------------------------*/
/* USER CODE BEGIN Application */

// Write to a register
void Lora_WriteReg(uint8_t addr, uint8_t val) {
    uint8_t data[2] = { (uint8_t)(addr | 0x80), val };
    // Cast and format for SPI
```

```
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin,
    GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi2, data, 2, 100);
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin,
    GPIO_PIN_SET);
}

// Read from a register
uint8_t Lora_ReadReg(uint8_t addr) {
    uint8_t addr_byte = addr & 0x7F; // MSB low for Read
    uint8_t val = 0;
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin,
    GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi2, &addr_byte, 1, 100);
    HAL_SPI_Receive(&hspi2, &val, 1, 100);
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin,
    GPIO_PIN_SET);
    return val;
}

// This function automatically fires the exact
millisecond a packet leaves the antenna or a timeout
occurs
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == LORA_DIO0_Pin || GPIO_Pin ==
    LORA_DIO1_Pin)
    {
        // Explicitly drop Channel 2 (Debug_IO) the
        microsecond the radio fires
        HAL_GPIO_WritePin(Timing_IO_GPIO_Port,
        Timing_IO_Pin, GPIO_PIN_RESET);
    }

    if (GPIO_Pin == LORA_DIO0_Pin) // Check if the
    interrupt came from PE0 (RxDone / TxDone)
    {
```

```
        if (DIO0_Line_current_state ==
        STATE_WAITING_FOR_TX)
        {

            // Drive gpio pin high - this is our 0s
            timing point
            HAL_GPIO_WritePin(Debug_IO_GPIO_Port,
            Debug_IO_Pin, GPIO_PIN_SET);

            // Start your microsecond stopwatch right
            here
            __HAL_TIM_SET_COUNTER(&htim5, 0);
            HAL_TIM_Base_Start(&htim5);
        }

        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        // ONLY notify the waiting FreeRTOS task. Do no
        SPI, do no UART!
        if (LoRaTaskHandle != NULL) {
            vTaskNotifyGiveFromISR(LoRaTaskHandle, &
            xHigherPriorityTaskWoken);
        }

        // Force FreeRTOS to instantly switch to the
        LoRa task if it has high priority
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
    // --- ADD THIS NEW BLOCK TO CATCH THE TIMEOUT ---
    else if (GPIO_Pin == LORA_DIO1_Pin) // Check if
    the interrupt came from PE1 (RxTimeout)
    {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        if (LoRaTaskHandle != NULL) {
            vTaskNotifyGiveFromISR(LoRaTaskHandle, &
            xHigherPriorityTaskWoken);
```

```
        }

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}


/**
 * @brief  Reads back the contents of the radio's TX FIFO to verify exactly
 *         what bytes are staged for transmission.
 * @param  buffer: Pointer to an array where the read-back bytes will be stored
 * @param  length: Total number of bytes to read back (should equal your 'idx' variable)
 */
void Lora_ReadFIFOVerify(uint8_t *buffer, uint8_t length)
{
    uint8_t addr_byte = 0x00 & 0x7F; // REG_FIFO (0x00) with Read Bit (MSB = 0)

    // Use your actual working LoRa CS labels suggested by the compiler
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_RESET);

    // Send register address byte over your operational SPI link
    HAL_SPI_Transmit(&hspi2, &addr_byte, 1, HAL_MAX_DELAY);

    // Read the array block out of the radio memory
    HAL_SPI_Receive(&hspi2, buffer, length, HAL_MAX_DELAY);

    // Pull CS back High to end transaction
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin,
```

```
    GPIO_PIN_SET);
}


// Pure integer frequency configuration math
void Lora_SetFrequency(uint32_t frequency_hz)
{
    uint32_t frf = (uint32_t)(((uint64_t)frequency_hz *
    16384) / 1000000);
    Lora_WriteReg(REG_FRF_MSB, (uint8_t)((frf >> 16) &
    0xFF));
    Lora_WriteReg(REG_FRF_MID, (uint8_t)((frf >> 8)  &
    0xFF));
    Lora_WriteReg(REG_FRF_LSB, (uint8_t)(frf         &
    0xFF));
}

#define REG_FIFO           0x00
#define REG_FIFO_ADDR_PTR  0x0D
#define REG_FIFO_RX_CURR   0x10
#define REG_RX_NB_BYTES    0x13

void LoRa_Extract_FIFO_Payload(uint8_t *rx_buffer,
uint8_t *payload_length)
{
    // 1. Check how many bytes the gateway actually
    dropped into our radio
    *payload_length = Lora_ReadReg(REG_RX_NB_BYTES);

    // Safety check to ensure we don't overflow an array
    if (*payload_length == 0 || *payload_length > 64) {
        return;
    }

    // 2. Fetch the starting memory address of the last
    received packet
    uint8_t current_fifo_addr = Lora_ReadReg(
```

```
    REG_FIFO_RX_CURR);

    // 3. Force the internal SPI pointer to hop to that
    specific location
    Lora_WriteReg(REG_FIFO_ADDR_PTR, current_fifo_addr);

    // 4. Sequentially read the bytes directly out of
    the FIFO register stream
    for (uint8_t i = 0; i < *payload_length; i++) {
        rx_buffer[i] = Lora_ReadReg(REG_FIFO);
    }
}

/* USER CODE END Application */
```